Treasure Hunt Game Notebook

Read and Review Your Starter Code

The theme of this project is a popular treasure hunt game in which the player needs to find the treasure before the pirate does. While you will not be developing the entire game, you will write the part of the game that represents the intelligent agent, which is a pirate in this case. The pirate will try to find the optimal path to the treasure using deep Q-learning.

To Begin: Use this **TreasureHuntGame_starterCode.ipynb** file to complete your assignment.

You have been provided with two Python classes and this notebook to help you with this assignment. The first class, **TreasureMaze.py**, represents the environment, which includes a maze object defined as a matrix. The second class, **GameExperience.py**, stores the episodes – that is, all the states that come in between the initial state and the terminal state. This is later used by the agent for learning by experience, called "exploration". This notebook shows how to play a game. Your task is to complete the deep Q-learning implementation in the qtrain() function for which a skeleton implementation has been provided.

NOTE: The code block you will need to complete will have **#TODO** as a header.
First, read and review the next few code and instruction blocks to understand the code that you have been given.

Installations The following command will install the necessary Python libraries to necessary to run this application. If you see a "[notice] A new release of pip is available: 23.1.2 -> 25.2" at the end of the installation, you may disregard that statement.

In [16]:
```
!pip install -r requirements.txt
```

```
Requirement already satisfied: numpy==1.26.4 in .\.venv\lib\site-packages (from -r r
equirements.txt (line 1)) (1.26.4)
Requirement already satisfied: keras in .\.venv\lib\site-packages (from -r requireme
nts.txt (line 2)) (3.15.1)
Requirement already satisfied: absl-py in .\.venv\lib\site-packages (from keras->-r
requirements.txt (line 2)) (2.5.0)
Requirement already satisfied: rich in .\.venv\lib\site-packages (from keras->-r req
uirements.txt (line 2)) (15.0.0)
Requirement already satisfied: namex in .\.venv\lib\site-packages (from keras->-r re
quirements.txt (line 2)) (0.1.0)
Requirement already satisfied: h5py in .\.venv\lib\site-packages (from keras->-r req
uirements.txt (line 2)) (3.14.0)
Requirement already satisfied: optree in .\.venv\lib\site-packages (from keras->-r r
equirements.txt (line 2)) (0.20.0)
Requirement already satisfied: ml-dtypes in .\.venv\lib\site-packages (from keras->-
r requirements.txt (line 2)) (0.5.4)
Requirement already satisfied: packaging in .\.venv\lib\site-packages (from keras->-
r requirements.txt (line 2)) (26.3)
Requirement already satisfied: typing-extensions>=4.10.0 in .\.venv\lib\site-package
s (from optree->keras->-r requirements.txt (line 2)) (4.16.0)
Requirement already satisfied: markdown-it-py>=2.2.0 in .\.venv\lib\site-packages (f
rom rich->keras->-r requirements.txt (line 2)) (4.2.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in .\.venv\lib\site-packages
(from rich->keras->-r requirements.txt (line 2)) (2.21.0)
Requirement already satisfied: mdurl~=0.1 in .\.venv\lib\site-packages (from markdow
n-it-py>=2.2.0->rich->keras->-r requirements.txt (line 2)) (0.1.2)
[notice] A new release of pip is available: 24.0 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

Tensorflow CPU Acceleration Warning

GPU/CUDA/Memory Warnings/Errors: You may receive some errors referencing that GPUs will not be used, CUDA could not be found, or free system memory allocation errors. These and a few others, are standard errors that can be ignored here as they are environment based.

Example messages:

- oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders
- WARNING: All log messages before absl::InitializeLog() is called are written to STDERR

In [17]:
```python
from __future__ import print_function
import os, sys, time, datetime, json, random
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import clone_model
from keras.models import Sequential
from keras.layers import Dense, Activation, PReLU
```

```
from keras.optimizers import SGD , Adam, RMSprop
import matplotlib.pyplot as plt
from TreasureMaze import TreasureMaze
from GameExperience import GameExperience
%matplotlib inline
```

Maze Object Generation

NOTE: The following code block contains an 8x8 matrix that will be used as a maze object:

In [18]:
```
maze = np.array([
    [ 1.,  0.,  1.,  1.,  1.,  1.,  1.,  1.],
    [ 1.,  0.,  1.,  1.,  1.,  0.,  1.,  1.],
    [ 1.,  1.,  1.,  1.,  0.,  1.,  0.,  1.],
    [ 1.,  1.,  1.,  0.,  1.,  1.,  1.,  1.],
    [ 1.,  1.,  0.,  1.,  1.,  1.,  1.,  1.],
    [ 1.,  1.,  1.,  0.,  1.,  0.,  0.,  0.],
    [ 1.,  1.,  1.,  0.,  1.,  1.,  1.,  1.],
    [ 1.,  1.,  1.,  1.,  0.,  1.,  1.,  1.]
])
```

Helper Functions and Global Variables

This **show()** helper function allows a visual representation of the maze object:

In [19]:
```
def show(qmaze):
    plt.grid('on')
    nrows, ncols = qmaze.maze.shape
    ax = plt.gca()
    ax.set_xticks(np.arange(0.5, nrows, 1))
    ax.set_yticks(np.arange(0.5, ncols, 1))
    ax.set_xticklabels([])
    ax.set_yticklabels([])
    canvas = np.copy(qmaze.maze)
    for row,col in qmaze.visited:
        canvas[row,col] = 0.6
    pirate_row, pirate_col, _ = qmaze.state
    canvas[pirate_row, pirate_col] = 0.3   # pirate cell
    canvas[nrows-1, ncols-1] = 0.9 # treasure cell
    img = plt.imshow(canvas, interpolation='none', cmap='gray')
    return img
```

The **pirate agent** can move in four directions: left, right, up, and down.

Note: While the agent primarily learns by experience through exploitation, often, the agent can choose to explore the environment to find previously undiscovered paths. This

is called "exploration" and is defined by epsilon. This value is the **EXPLORATION** values from the Cartpole assignment. The hyperparameters are provided here and used in the **qtrain()** method. You are encouraged to try various values for the exploration factor and see how the algorithm performs.

In [20]:
```
LEFT = 0
UP = 1
RIGHT = 2
DOWN = 3


# Exploration factor
epsilon = 0.1

# Actions dictionary
actions_dict = {
    LEFT: 'left',
    UP: 'up',
    RIGHT: 'right',
    DOWN: 'down',
}

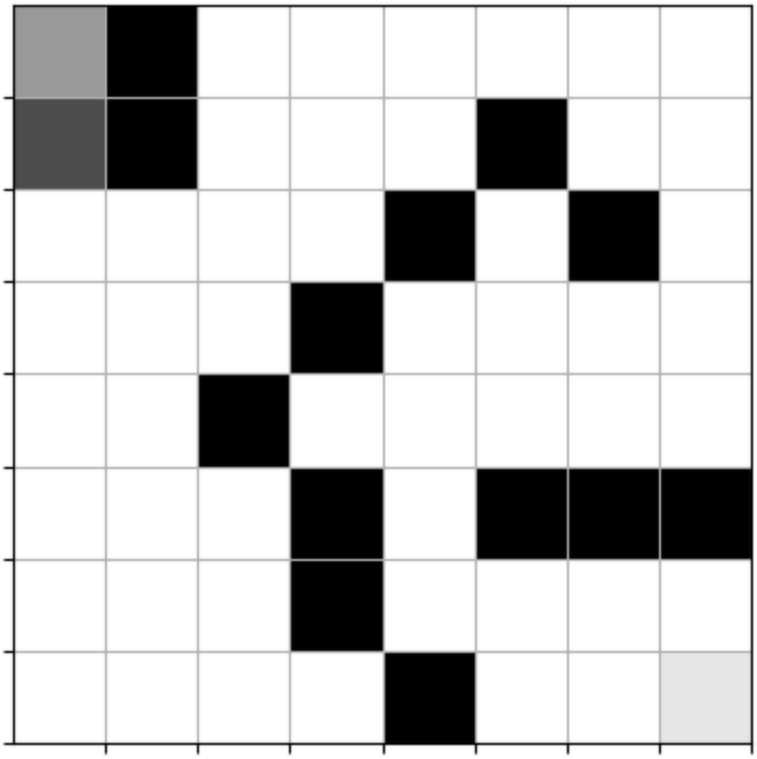
num_actions = len(actions_dict)
```

The sample code block and output below show creating a maze object and performing one action (DOWN), which returns the reward. The resulting updated environment is visualized.

In [21]:
```
qmaze = TreasureMaze(maze)
canvas, reward, game_over = qmaze.act(DOWN)
print("reward=", reward)
show(qmaze)
```

```
reward= -0.04
```

Out[21]:
```
<matplotlib.image.AxesImage at 0x181cf582210>
```



NOTE: This **play_game()** function simulates a full game based on the provided trained model. The other parameters include the TreasureMaze object, the starting position of the pirate and max amount of steps to make sure the code does not get stuck in a loop.

In [22]:
```
def play_game(model, qmaze, pirate_cell, max_steps=None):
    qmaze.reset(pirate_cell)
    envstate = qmaze.observe()
    steps = 0
    if max_steps is None:
        max_steps = qmaze.maze.size * 4  # safety cutoff

    while steps < max_steps:
        state = np.asarray(envstate, dtype=np.float32)
        if state.ndim == 1:
            state = np.expand_dims(state, axis=0)

        q_values = model(state, training=False).numpy()[0]
        valid_actions = qmaze.valid_actions()

        if len(valid_actions) == 0:
            return False

        valid_q = {a: q_values[a] for a in valid_actions}
        action = max(valid_q, key=valid_q.get)

        envstate, reward, game_status = qmaze.act(action)
        steps += 1

        if game_status == 'win':
```

```
            return True
        elif game_status == 'lose':
            return False

    return False  # timed out with no result
```

Note: This **completion_check()** function helps you to determine whether the pirate can win any game at all. If your maze is not well designed, the pirate may not win any game at all. In this case, your training would not yield any result. The provided maze in this notebook ensures that there is a path to win and you can run this method to check.

In [23]:
```
def completion_check(model, maze_or_qmaze, max_steps=None):
    # Accept either raw numpy maze or TreasureMaze instance
    if isinstance(maze_or_qmaze, TreasureMaze):
        qmaze = maze_or_qmaze
    else:
        qmaze = TreasureMaze(maze_or_qmaze)

    for cell in qmaze.free_cells:
        if not qmaze.valid_actions(cell):
            continue
        if not play_game(model, qmaze, cell, max_steps=max_steps):
            return False
    return True
```

Note: The **build_model()** function in the block below will build the neural network model. Review the code and note the number of layers, as well as the activation, optimizer, and loss functions that are used to train the model.

In [24]:
```
def build_model(maze):
    model = Sequential()
    model.add(Dense(maze.size, input_shape=(maze.size,)))
    model.add(PReLU())
    model.add(Dense(maze.size))
    model.add(PReLU())
    model.add(Dense(num_actions))
    model.compile(optimizer='adam', loss='mse')
    return model
```

Note: This **train_step()** helper function in the block below is used to help predict Q-values (quality values) in the current modelto see how good each action is in a given state and improve the Q-network by reducing the gap between what is predicted and what should have been predicted.

If you're interested in reading up on the *@tf.function*, which is a decorator for Tensorflow to run this code into a TensorFlow computation graph, please refer to this link:

https://www.tensorflow.org/guide/intro_to_graphs

Tensorflow GPU Warning

You will see a **warning in red** "INTERNAL: CUDA Runtime error: Failed call to cudaGetRuntimeVersion: Error loading CUDA libraries. GPU will not be used.". This is simply coming from **Tensorflow skipping using GPU for this assignment.**

In [25]:
```
loss_fn = tf.keras.losses.MeanSquaredError()
optimizer = tf.keras.optimizers.Adam()

@tf.function
def train_step(x, y):
    with tf.GradientTape() as tape:
        q_values = model(x, training=True)
        loss = loss_fn(y, q_values)
    grads = tape.gradient(loss, model.trainable_variables)
    optimizer.apply_gradients(zip(grads, model.trainable_variables))
    return loss
```

#TODO: Complete the Q-Training Algorithm Code Block

This is your deep Q-learning implementation. The goal of your deep Q-learning implementation is to find the best possible navigation sequence that results in reaching the treasure cell while maximizing the reward. In your implementation, you need to determine the optimal number of epochs to achieve a 100% win rate.

Pseudocode:
For each epoch: Reset the environment at a random starting cell agent_cell = randomly select a free cell
Hint: Review the reset method in the TreasureMaze.py class.

```
        Set the initial environment state
        env_state should reference the environment's current state
        Hint: Review the observe method in the TreasureMaze.py class.

        While game status is not game over:
           previous_envstate = env_state
            Decide on an action:
                - If possible, take a random valid exploration action
    and
                  randomly choose action (left, right, up, down)
                  and assign it to an action variable
```

```
            - Else, pick the best exploitation action from the
    model and assign it to an action variable
                Hint: Review the predict method in the
    GameExperience.py class.

           Retrieve the values below from the act() method.
           env_state, reward, game_status = qmaze.act(action)
           Hint: Review the act method in the TreasureMaze.py class.

            Track the wins and losses from the game_status using
    win_history

           Store the episode below in the Experience replay object
           episode = [previous_envstate, action, reward, envstate,
    game_status]
           Hint: Review the remember method in the GameExperience.py
    class.

           Train neural network model and evaluate loss
           Hint: Call GameExperience.get_data to retrieve training data
    (input and target)
           and pass to the train_step method and assign it to
    batch_loss and append to the loss variable

      If the win rate is above the threshold and your model passes the
    completion check, that would be your epoch.
```

Note: A 100% win rate **DOES NOT EXPLICITLY MEAN** that you have solved the maze. It simply indicates that during the last evaluation, the pirate *happened* to get to the treasure. Be sure to utilise the **completion_check()** function to validate your pirate found the treasure at every starting point and consistently!

You will need to complete the section starting with #START_HERE. Please use the pseudocode above as guidance.

In [26]:
```python
def qtrain(model, maze, **opt):
    # exploration factor
    global epsilon

    # Number of epochs
    n_epoch = opt.get('n_epoch', 15000)

    # Maximum meory to store episodes
    max_memory = opt.get('max_memory', 1000)

    # Maximum data size for training
    data_size = opt.get('data_size', 50)

    # Frequency of target network updates
    target_update_freq = opt.get('target_update_freq', 50)

    # Start time
```

```
    start_time = datetime.datetime.now()

    # Construct environment/game from numpy array: maze (see argument above)
    qmaze = TreasureMaze(maze)

    # Target Network to better guide training
    target_model = clone_model(model)
    target_model.set_weights(model.get_weights())

    # Initialize experience replay object
    experience = GameExperience(model, target_model, max_memory=max_memory)

    win_history = [] # history of win/lose game
    hsize = qmaze.maze.size // 2 #history window size
    win_rate = 0.0
    decayRate = 0.1

    # ============START_HERE================
    for epoch in range(n_epoch):
        # Reset environment at random starting location
        Agent_cell = random.choice(qmaze.free_cells)
        qmaze.reset(Agent_cell)

        # Get initial state
        envstate = qmaze.observe()

        n_episodes = 0
        loss = 0.0
        gameOver = False

        while not gameOver:
            # Save current state
            previous_envstate = envstate

            # Get valid moves
            validAction = qmaze.valid_actions()

            if len(validAction) == 0:
                break

            # Exploration vs exploitation
            if np.random.rand() < epsilon:
                # Explore
                Action = random.choice(validAction)

            else:
                # Exploit learned knowledge
                q_values = experience.predict(previous_envstate)

                # Only choose valid moves
                valid_q = {
                    a: q_values[a] for a in validAction
                }

                Action = max(valid_q, key=valid_q.get)
```

```
            # Take action
            env_state, reward, game_status = qmaze.act(Action)

            # update current environment state
            envstate = env_state

            # Store correct experience:
            # previous state -> action -> reward -> new state
            episode = [
                previous_envstate,
                Action,
                reward,
                env_state,
                game_status
            ]

            n_episodes += 1

            experience.remember(episode)

            # Train model
            inputs, targets = experience.get_data(data_size)

            if inputs is not None and len(inputs) > 0:

                history = model.fit(
                    inputs,
                    targets,
                    epochs=1,
                    batch_size=16,
                    verbose=0
                )

                loss = history.history['loss'][0]

            # Track wins/losses
            if game_status == 'win':
                win_history.append(1)
                gameOver = True

            elif game_status == 'lose':
                win_history.append(0)
                gameOver = True

            else:
                gameOver = False

        # Calculate win rate
        if len(win_history) >= hsize:
            win_rate = sum(win_history[-hsize:]) / hsize

        else:
            win_rate = 0.0

        # Print progress
        dt = datetime.datetime.now() - start_time
```

```
        t = format_time(
            dt.total_seconds()
        )

        template = (
            "Epoch: {:03d}/{:d} | "
            "Loss: {:.4f} | "
            "Episodes: {:d} | "
            "Win count: {:d} | "
            "Win rate: {:.3f} | "
            "time: {}"
        )

        print(
            template.format(
                epoch,
                n_epoch-1,
                loss,
                n_episodes,
                sum(win_history),
                win_rate,
                t
            )
        )

        # Update the target model periodically to provide more stable Q-value targe
        if epoch % target_update_freq == 0:
            target_model.set_weights(model.get_weights())

        # Reduce exploration
        if win_rate > 0.9:
            epsilon = 0.05

        # Completion check
        if win_rate >= 0.999:
            result = completion_check(
                model,
                TreasureMaze(maze)
            )
            if result:
                print("Reached 100%% win rate at epoch: %d" % epoch)
                break

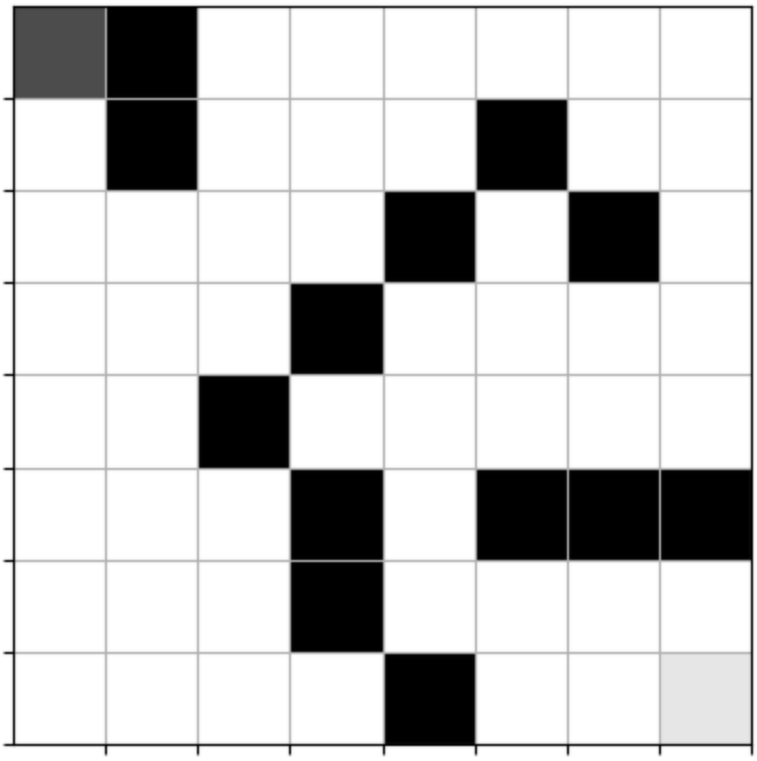
# This is a small utility for printing readable time strings:
def format_time(seconds):
    if seconds < 400:
        s = float(seconds)
        return "%.1f seconds" % (s,)
    elif seconds < 4000:
        m = seconds / 60.0
        return "%.2f minutes" % (m,)
    else:
        h = seconds / 3600.0
        return "%.2f hours" % (h,)
```

Test Your Model

Now we will start testing the deep Q-learning implementation. To begin, select **Cell**, then **Run All** from the menu bar. This will run your notebook. As it runs, you should see output begin to appear beneath the next few cells. The code below creates an **instance** of TreasureMaze. This does not show your actual training done.

In [27]:
```
qmaze = TreasureMaze(maze)
show(qmaze)
```

Out[27]: <matplotlib.image.AxesImage at 0x181cb321150>



In the next code block, you will build your model using the **build_model** function and train it using deep Q-learning. Note: This step takes several minutes to fully run.

WARNING If you did not attempt the assignment, the code **will** error out at this section.

In [28]:
```
model = build_model(maze)
qtrain(model, maze, n_epoch=1000, max_memory=8*maze.size, data_size=32, target_upda
```

```
c:\pirate-intelligent-agent\.venv\Lib\site-packages\keras\src\layers\core\dense.py:1
07: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When
using Sequential models, prefer using an `Input(shape)` object as the first layer in
the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

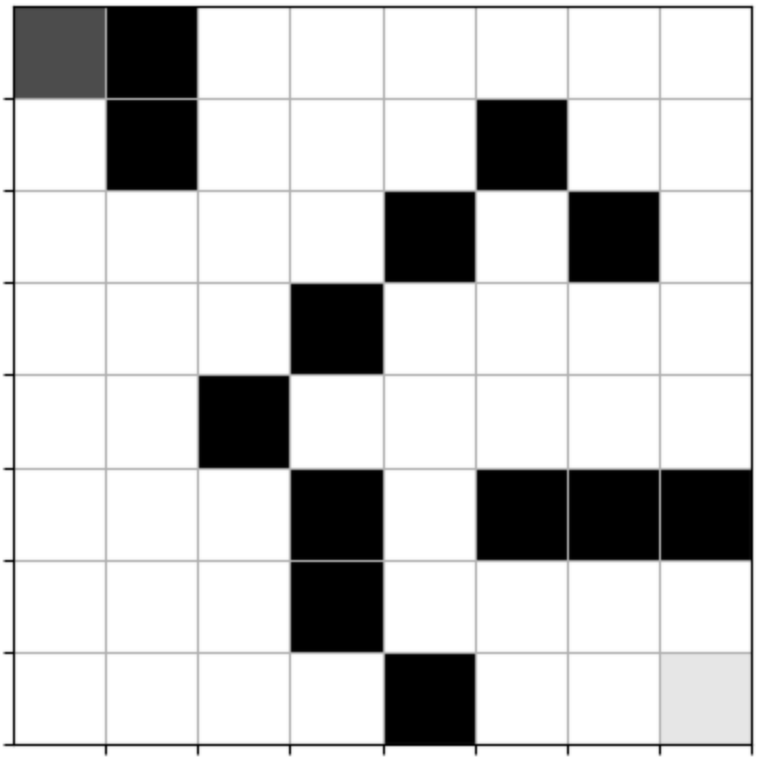
```
Epoch: 000/999 | Loss: 0.0009 | Episodes: 149 | Win count: 0 | Win rate: 0.000 | tim
e: 10.5 seconds
Epoch: 001/999 | Loss: 0.0020 | Episodes: 67 | Win count: 1 | Win rate: 0.000 | tim
e: 14.9 seconds
Epoch: 002/999 | Loss: 0.0017 | Episodes: 144 | Win count: 1 | Win rate: 0.000 | tim
e: 24.2 seconds
Epoch: 003/999 | Loss: 0.0010 | Episodes: 61 | Win count: 2 | Win rate: 0.000 | tim
e: 28.1 seconds
Epoch: 004/999 | Loss: 0.0010 | Episodes: 150 | Win count: 3 | Win rate: 0.000 | tim
e: 37.6 seconds
Epoch: 005/999 | Loss: 0.0020 | Episodes: 85 | Win count: 4 | Win rate: 0.000 | tim
e: 42.9 seconds
Epoch: 006/999 | Loss: 0.0076 | Episodes: 43 | Win count: 5 | Win rate: 0.000 | tim
e: 45.6 seconds
Epoch: 007/999 | Loss: 0.0015 | Episodes: 97 | Win count: 6 | Win rate: 0.000 | tim
e: 51.8 seconds
Epoch: 008/999 | Loss: 0.0068 | Episodes: 6 | Win count: 7 | Win rate: 0.000 | time:
52.1 seconds
Epoch: 009/999 | Loss: 0.0021 | Episodes: 17 | Win count: 8 | Win rate: 0.000 | tim
e: 53.2 seconds
Epoch: 010/999 | Loss: 0.0027 | Episodes: 63 | Win count: 9 | Win rate: 0.000 | tim
e: 57.2 seconds
Epoch: 011/999 | Loss: 0.0017 | Episodes: 10 | Win count: 10 | Win rate: 0.000 | tim
e: 57.8 seconds
Epoch: 012/999 | Loss: 0.0028 | Episodes: 43 | Win count: 11 | Win rate: 0.000 | tim
e: 60.6 seconds
Epoch: 013/999 | Loss: 0.0021 | Episodes: 35 | Win count: 12 | Win rate: 0.000 | tim
e: 62.8 seconds
Epoch: 014/999 | Loss: 0.0020 | Episodes: 17 | Win count: 13 | Win rate: 0.000 | tim
e: 63.9 seconds
Epoch: 015/999 | Loss: 0.0091 | Episodes: 17 | Win count: 14 | Win rate: 0.000 | tim
e: 64.9 seconds
Epoch: 016/999 | Loss: 0.0019 | Episodes: 35 | Win count: 15 | Win rate: 0.000 | tim
e: 67.2 seconds
Epoch: 017/999 | Loss: 0.0023 | Episodes: 26 | Win count: 16 | Win rate: 0.000 | tim
e: 68.8 seconds
Epoch: 018/999 | Loss: 0.0018 | Episodes: 1 | Win count: 17 | Win rate: 0.000 | tim
e: 68.9 seconds
Epoch: 019/999 | Loss: 0.0015 | Episodes: 25 | Win count: 18 | Win rate: 0.000 | tim
e: 70.5 seconds
Epoch: 020/999 | Loss: 0.0016 | Episodes: 18 | Win count: 19 | Win rate: 0.000 | tim
e: 71.6 seconds
Epoch: 021/999 | Loss: 0.0013 | Episodes: 22 | Win count: 20 | Win rate: 0.000 | tim
e: 73.0 seconds
Epoch: 022/999 | Loss: 0.0015 | Episodes: 11 | Win count: 21 | Win rate: 0.000 | tim
e: 73.7 seconds
Epoch: 023/999 | Loss: 0.0020 | Episodes: 28 | Win count: 22 | Win rate: 0.000 | tim
e: 75.5 seconds
Epoch: 024/999 | Loss: 0.0029 | Episodes: 23 | Win count: 23 | Win rate: 0.000 | tim
e: 77.0 seconds
Epoch: 025/999 | Loss: 0.0015 | Episodes: 39 | Win count: 24 | Win rate: 0.000 | tim
e: 79.5 seconds
Epoch: 026/999 | Loss: 0.0015 | Episodes: 25 | Win count: 25 | Win rate: 0.000 | tim
e: 81.1 seconds
Epoch: 027/999 | Loss: 0.0060 | Episodes: 23 | Win count: 26 | Win rate: 0.000 | tim
e: 82.6 seconds
```

```
Epoch: 028/999 | Loss: 0.0026 | Episodes: 32 | Win count: 27 | Win rate: 0.000 | tim
e: 84.6 seconds
Epoch: 029/999 | Loss: 0.0016 | Episodes: 9 | Win count: 28 | Win rate: 0.000 | tim
e: 85.2 seconds
Epoch: 030/999 | Loss: 0.0013 | Episodes: 28 | Win count: 29 | Win rate: 0.000 | tim
e: 87.1 seconds
Epoch: 031/999 | Loss: 0.0009 | Episodes: 7 | Win count: 30 | Win rate: 0.938 | tim
e: 87.7 seconds
Epoch: 032/999 | Loss: 0.0014 | Episodes: 17 | Win count: 31 | Win rate: 0.969 | tim
e: 88.8 seconds
Epoch: 033/999 | Loss: 0.0036 | Episodes: 14 | Win count: 32 | Win rate: 0.969 | tim
e: 89.8 seconds
Epoch: 034/999 | Loss: 0.0009 | Episodes: 11 | Win count: 33 | Win rate: 1.000 | tim
e: 90.5 seconds
Epoch: 035/999 | Loss: 0.0012 | Episodes: 31 | Win count: 34 | Win rate: 1.000 | tim
e: 93.1 seconds
Epoch: 036/999 | Loss: 0.0052 | Episodes: 23 | Win count: 35 | Win rate: 1.000 | tim
e: 95.0 seconds
Epoch: 037/999 | Loss: 0.0017 | Episodes: 16 | Win count: 36 | Win rate: 1.000 | tim
e: 96.6 seconds
Epoch: 038/999 | Loss: 0.0033 | Episodes: 31 | Win count: 37 | Win rate: 1.000 | tim
e: 99.0 seconds
Epoch: 039/999 | Loss: 0.0012 | Episodes: 13 | Win count: 38 | Win rate: 1.000 | tim
e: 100.3 seconds
Epoch: 040/999 | Loss: 0.0041 | Episodes: 17 | Win count: 39 | Win rate: 1.000 | tim
e: 101.8 seconds
Epoch: 041/999 | Loss: 0.0011 | Episodes: 3 | Win count: 40 | Win rate: 1.000 | tim
e: 102.4 seconds
Epoch: 042/999 | Loss: 0.0023 | Episodes: 15 | Win count: 41 | Win rate: 1.000 | tim
e: 103.8 seconds
Epoch: 043/999 | Loss: 0.0014 | Episodes: 27 | Win count: 42 | Win rate: 1.000 | tim
e: 106.2 seconds
Epoch: 044/999 | Loss: 0.0019 | Episodes: 56 | Win count: 43 | Win rate: 1.000 | tim
e: 110.2 seconds
Epoch: 045/999 | Loss: 0.0016 | Episodes: 10 | Win count: 44 | Win rate: 1.000 | tim
e: 111.2 seconds
Epoch: 046/999 | Loss: 0.0007 | Episodes: 23 | Win count: 45 | Win rate: 1.000 | tim
e: 113.1 seconds
Epoch: 047/999 | Loss: 0.0010 | Episodes: 5 | Win count: 46 | Win rate: 1.000 | tim
e: 114.0 seconds
Epoch: 048/999 | Loss: 0.0030 | Episodes: 27 | Win count: 47 | Win rate: 1.000 | tim
e: 116.2 seconds
Epoch: 049/999 | Loss: 0.0009 | Episodes: 24 | Win count: 48 | Win rate: 1.000 | tim
e: 118.2 seconds
Epoch: 050/999 | Loss: 0.0002 | Episodes: 11 | Win count: 49 | Win rate: 1.000 | tim
e: 119.4 seconds
Reached 100% win rate at epoch: 50
```

Note: This cell will check to see if the model passes the completion check. Note: This could take several minutes.

In [29]:
```
completion_check(model, maze)
show(qmaze)
```

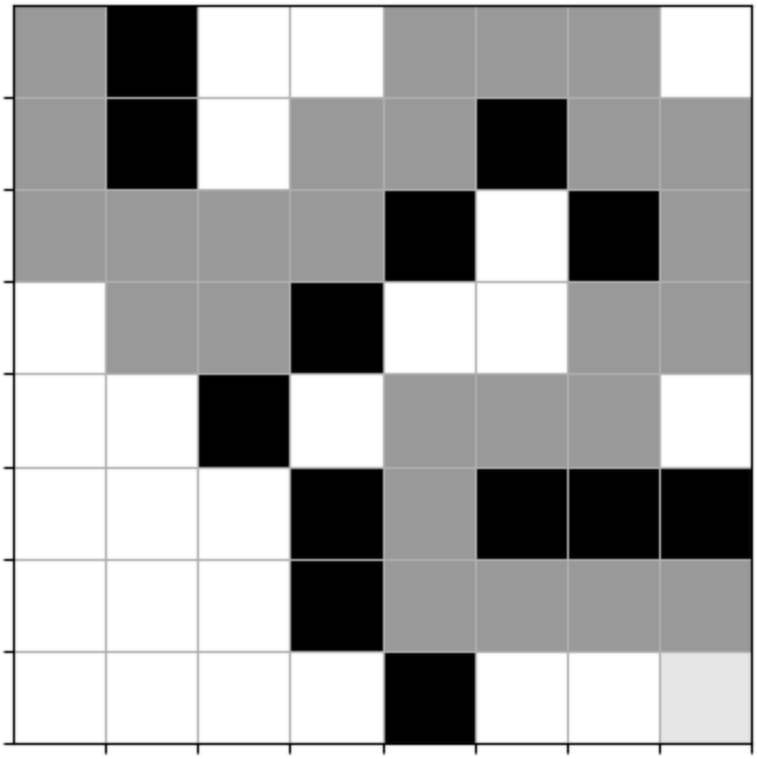
Out[29]: <matplotlib.image.AxesImage at 0x181ced41e90>



This cell will test your model for one game. It will start the pirate at the top-left corner and run **play_game()**. The agent should find a path from the starting position to the target (treasure). The treasure is located in the bottom-right corner.

In [30]:
```
pirate_start = (0, 0)
play_game(model, qmaze, pirate_start)
show(qmaze)
```

Out[30]: <matplotlib.image.AxesImage at 0x181cfed4950>



Save and Submit Your Work

Hint: To use the markdown block below, double click in the **Type Markdown and LaTeX: α2** block below, to turn it back to html, Run the cell.

After you have finished creating the code for your notebook, save your work. Make sure that your notebook contains your name in the filename (e.g. Doe_Jane_ProjectTwo.html). Download this file as an .html file clicking on ***file*** in *Jupyter Notebook*, navigating down to ***Download as*** and clicking on ***.html***. Download a copy of your .html file and submit it to Brightspace.

In [ ]: